Campus Recruitment Management API

A placement-cell REST API that enforces eligibility, a status workflow and offer limits, and stays correct under retries and races.

Kriti Singhal · Technical documentation · October 2026

GitHub: github.com/krit566/campus-recruitment-api

Contents

1. Overview

2. Architecture

3. Data model

4. Rules the API enforces

5. Walkthrough: real requests

6. Reliability

7. Security and observability

8. Testing

9. Limitations and next steps

10. Interview questions

1 Overview

A REST API for a college placement cell: students, companies, job postings and applications. Beyond basic create/read/update, it enforces the rules a real placement process depends on (**eligibility, an application workflow, an offer limit**) and it stays correct when things go wrong: **clients retry, two reviewers click at the same moment, a student tries to act as someone else**.

19
endpoints, with OpenAPI docs

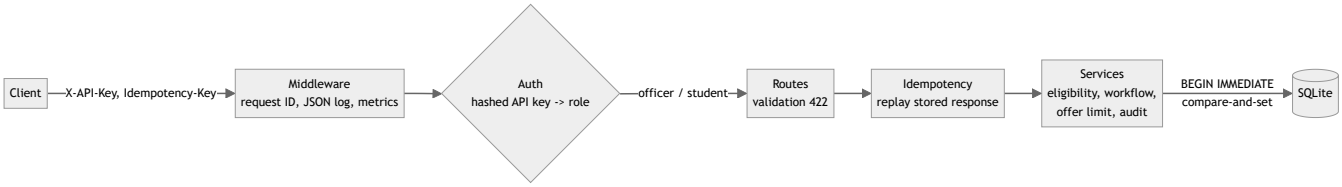
1
winner when 8 parallel offers race for 1 opening

2
roles (officer, student), checked on every route

48
automated tests in CI

Python · FastAPI · SQLite with plain SQL (no ORM) · Pydantic · pytest · Docker · GitHub Actions · Render blueprint

2 Architecture



Layer	File	Knows about
HTTP	`app/main.py`	Routes, status codes, request and response models
Validation	`app/schemas.py`	Email, CGPA 0–10, branch codes, roll-number format, deadlines
Business rules	`app/services.py`	Eligibility, workflow, offer limit, audit events
Data	`app/repository.py`	SQL only
Cross-cutting	`security.py`, `idempotency.py`, `observability.py`	Keys and roles, retry safety, request IDs and metrics

Campus Recruitment Management API 2.0.0 OAS 3.1

/openapi.json

meta

- GET /health Health
- GET /metrics Metrics

students

- POST /students Create Student
- GET /students List Students
- GET /students/{sid} Get Student
- PATCH /students/{sid} Update Student
- DELETE /students/{sid} Delete Student
- GET /students/{sid}/eligible-jobs Eligible Jobs

companies

- POST /companies Create Company
- GET /companies List Companies

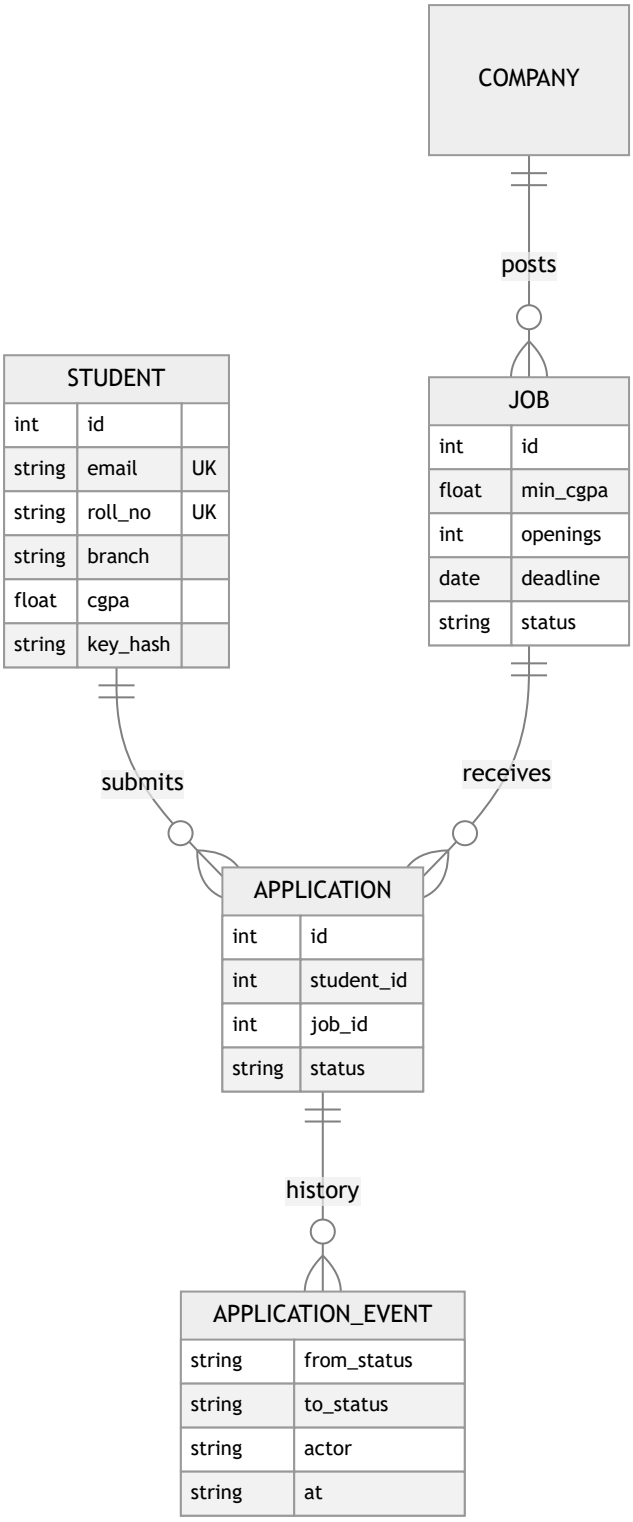
jobs

- POST /jobs Create Job
- GET /jobs List Jobs
- GET /jobs/{jid} Get Job
- POST /jobs/{jid}/close Close Job
- GET /jobs/{jid}/stats Job Stats

applications

- POST /applications Apply
- GET /applications List Applications
- PATCH /applications/{aid}/status Change Status

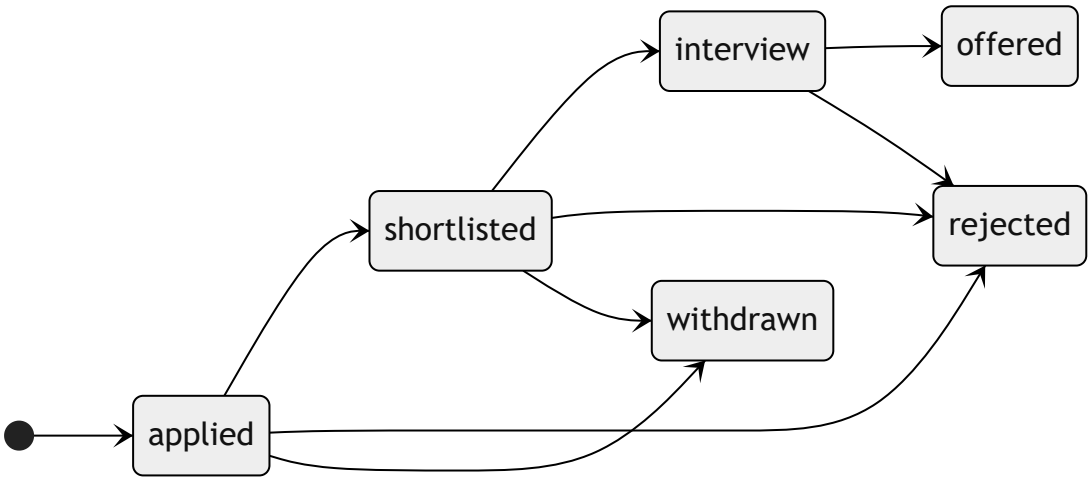
3 Data model



One application per student per job is a database `UNIQUE` constraint, so it holds even when two requests arrive together; a Python check alone could be raced.

4 Rules the API enforces

Rule	Response when broken
Valid input (email, CGPA 0–10, known branch, openings > 0, deadline not in the past)	422
Unique email, roll number, company name; one application per student per job	409
Branch eligible and CGPA at least the job's minimum	403
Students only see and change their own records (not their CGPA)	403
Status moves only along the workflow below	409
No more offers than openings; the job closes itself when full	409



5 Walkthrough: real requests

Output of `scripts/walkthrough.py` against a fresh local server (a job with **one** opening, minimum CGPA 7.5, CSE/IT only):

```
POST /jobs  openings=1, min CGPA 7.5, CSE/IT
  -> 201  {"id": 1, "title": "SWE Intern", "openings": 1, "status": "open"}
POST /students x4  -> 201, each returns its own API key once (only a SHA-256 hash is stored)
Asha (student) applies, Idempotency-Key: a1
  -> 201  {"id": 1, "status": "applied"}
same request retried after a timeout (key a1)
  -> 201  Idempotent-Replayed: true  {"id": 1, "status": "applied"}
Meera (ME branch) applies
  -> 403  {"detail": "branch ME is not eligible for this job"}
Kabir (CGPA 6.9) applies
  -> 403  {"detail": "CGPA 6.9 is below the minimum 7.5"}
Ravi tries to apply as Asha
  -> 403  {"detail": "students can only access their own records"}
Officer: applied -> offered (skips interview)
  -> 409  {"detail": "cannot move an application from 'applied' to 'offered'"}
Two reviewers offer the last opening at the same moment
  -> [200, 409]  (exactly one 200, the other 409)
GET /jobs/1/stats
  -> 200  {"openings": 1, "total_applications": 2, "by_status": {"interview": 1, "offered": 1},
"offers_left": 0}
GET /jobs/1
  -> 200  {"status": "closed"}
```

6 Reliability

Retries: idempotency keys

A client that times out cannot know whether its request worked. With an `Idempotency-Key`, the first response is stored and a retry gets the same response back (`Idempotent-Replayed: true`) instead of acting twice. Reusing a key with a different body is rejected (422); failed requests are not stored, so a corrected retry still works. Payment APIs require the same pattern.

Races: the last opening

`change_status` reads the application, checks "offers left?" and writes inside one `BEGIN IMMEDIATE` transaction, which takes SQLite's write lock before reading. The second reviewer therefore sees the first offer and gets 409. The update is also compare-and-set (`WHERE status = <status we read>`). `tests/test_concurrency.py` fires 8 parallel offers at a job with one opening and checks that exactly one wins.

> Writing that test found a real bug: the database connection was opened on one worker thread and used on another, which SQLite refuses by default. The fix and the reasoning are in `app/db.py`.

7 Security and observability

Concern	How
Authentication	`X-API-Key` on every request. Student keys are random, shown once, stored as SHA-256 hashes; the officer key is compared in constant time.
Authorisation	Officer: everything. Student: own profile (not CGPA), own applications, job listings. Checked in routes and again in services.
Audit	Every status change records from, to, actor (`officer` / `student:<id>`) and time.
Tracing	`X-Request-ID` on every response (echoed if sent), one JSON log line per request.
Metrics	`/metrics` : requests, status codes and p50/p95 latency per route template (officer only).
Delivery	Dockerfile (non-root user, database in a volume), GitHub Actions CI, one-click Render blueprint.

8 Testing

Suite	Covers
`test_students.py`	Validation, uniqueness, filters, pagination, partial updates, delete cascade
`test_jobs_and_applications.py`	Eligibility, deadlines, workflow transitions, offer limit, auto-close, stats
`test_security_reliability.py`	Keys, roles, ownership, idempotent replay and conflicts, request IDs, metrics
`test_concurrency.py`	8 parallel offers for 1 opening: exactly one succeeds

```
pip install -r requirements.txt
$env:RECRUIT_ADMIN_KEY = "<a long random string>"
uvicorn app.main:app --reload        # http://127.0.0.1:8000/docs
python -m pytest -q                  # 48 tests
```

9 Limitations and next steps

- Idempotency is checked before and stored after the action, outside one transaction: two *simultaneous* first attempts with the same key could both run (the `UNIQUE` rule still blocks a duplicate application). Fix: reserve the key in the same transaction.
- Key rotation and expiry; per-key rate limiting.
- Migrations (Alembic) and PostgreSQL for several instances, with row locks (`SELECT … FOR UPDATE`) instead of SQLite's single write lock.
- Export metrics to Prometheus instead of in-process counters.

10 Interview questions

Why a database constraint and not only a Python check for duplicates?

Two requests can both pass the Python check before either inserts. The constraint is enforced by the database at write time.

Why `BEGIN IMMEDIATE` instead of `BEGIN`?

A plain `BEGIN` takes the write lock only at the first write, so two transactions could both read "1 offer left". `IMMEDIATE` locks before the read.

422 vs 403 vs 404 vs 409?

Malformed input; allowed input but a rule forbids it; record missing; request conflicts with the current state.

Why store API keys hashed and compare in constant time?

A leaked database does not leak usable keys; constant-time comparison stops timing attacks from guessing a key byte by byte.

What does an idempotency key protect against?

A client retrying after a timeout and causing the action twice: a second application, a second status change, or in payments a second charge.